

ConvoCore — Complete API Reference Guide



Authentication Endpoints

Base URL: `/api/v1/auth`

Method	Endpoint	Description	Auth Required
POST	<code>/register</code>	Registers a new user account.	✗
POST	<code>/login</code>	Authenticates a user and returns access/refresh tokens.	✗
POST	<code>/refresh</code>	Generates a new access token using a valid refresh token.	✗
POST	<code>/logout</code>	Invalidates the current refresh token (blacklisting).	✓
POST	<code>/guest</code>	Issues a guest token (no user registration required).	✗
GET	<code>/me</code>	Retrieves the profile details of the current authenticated user.	✓

JWT Token Structure

```
// Access Token – short-lived (15 minutes expiry)
{
  "sub": "uuid",           // Subject (User ID)
  "type": "access",       // Token type
  "iat": 1730000000,       // Issued at timestamp
  "exp": 1730000900       // Expiration timestamp
}

// Refresh Token – long-lived (7 days expiry, persisted in Redis)
{
  "sub": "uuid",           // Subject (User ID)
  "type": "refresh",      // Token type
  "iat": 1730000000,
  "exp": 1730604800
}
```



Conversation Endpoints

Base URL: `/api/v1/conversations`

Method	Endpoint	Description
POST	<code>/</code>	Initiates a new conversation (direct or group).
GET	<code>/</code>	Retrieves a list of the user's active conversations.
GET	<code>/:id</code>	Retrieves detailed information for a specific conversation.
DELETE	<code>/:id</code>	Deletes a conversation (creator only).
POST	<code>/:id/participants</code>	Adds a participant to the specified conversation.
DELETE	<code>/:id/participants/:userId</code>	Removes a participant from the specified conversation.

Create Conversation — Request Body Schema

```
{
  "type": "group",
  "name": "Project Team",
  "participantIds": ["uuid-1", "uuid-2"]
}
```

Authorization Rule: Access and modification of a conversation are restricted exclusively to its participants.



Message Endpoints

Base URL: `/api/v1/conversations/:id/messages`

Method	Endpoint	Description
POST	/	Submits a new message to the conversation.
GET	/	Retrieves a paginated list of messages within the conversation.
PATCH	/:messageId	Edits a message previously sent by the current user.
DELETE	/:messageId	Deletes a message previously sent by the current user.
POST	/read	Marks all unread messages in the conversation as read.

Send Message — Request Body Schema

```
{
  "content": "Hello world!"
}
```



Notification Endpoints

Base URL: `/api/v1/notifications`

Method	Endpoint	Description
GET	/	Retrieves a paginated list of notifications.
GET	/unread-count	Retrieves the total count of unread notifications.
PATCH	/:id/read	Marks a specific notification as read.
PATCH	/read-all	Marks all user notifications as read.
DELETE	/:id	Deletes a specific notification.



Presence Endpoints

Base URL: `/api/v1/presence`

Method	Endpoint	Description
--------	----------	-------------

POST	/online	Sets the current user's status to online.
POST	/offline	Sets the current user's status to offline.
POST	/typing	Broadcasts a typing indicator for a conversation.
GET	/:conversationId	Retrieves presence status of all conversation participants.

Search Endpoints

Base URL: `/api/v1/search`

Method	Endpoint	Description
GET	/	Global search across messages and conversations.
GET	/messages	Search specifically within accessible messages.
GET	/conversations	Search specifically within user's conversations.

Upload Endpoints

Base URL: `/api/v1/uploads`

Method	Endpoint	Description
POST	/image	Uploads an image file (PNG/JPG/GIF/WEBP).
POST	/file	Uploads a general document/file.
GET	/:fileId	Retrieves file metadata and access URL.
DELETE	/:fileId	Deletes an uploaded file (owner only).

Server-Sent Events (SSE) / Real-time

Endpoint: `GET /__transmit/events?token=<jwt>`

Subscription Endpoint: `POST /__transmit/subscribe`

Supported Channels

Channel Pattern	Events Broadcasted
conversations/:id	message:created, message:updated, message:deleted
users/:id/notifications	notification:created
conversations/:id/presence	presence:online, presence:offline, presence:typing

Security and Middleware Architecture

Request Processing Flow:

```
Request
→ RateLimiterMiddleware (Redis sliding window)
```

- SecurityHeadersMiddleware (CSP, HSTS, X-Frame-Options)
- AuthMiddleware (validates Bearer token)
- ConversationAccessMiddleware (verifies membership for /conversations/:id/* routes)
- RequestValidator (VineJS schema validation)
- Controller (HTTP routing)
- Service (Business logic)
- Repository (Lucid ORM / Database)

HTTP Status Code Mapping

Code	Scenario	Description
200	OK	Successful read, update, or deletion.
201	Created	Successful resource creation.
401	Unauthorized	Missing, invalid, or expired JWT.
403	Forbidden	Valid token, but insufficient permissions.
404	Not Found	Requested resource does not exist.
409	Conflict	Duplicate resource (e.g., existing email).
422	Unprocessable Entity	Request failed due to validation errors.
429	Too Many Requests	Rate limit exceeded.
500	Internal Server Error	System failure.

Database Schema Definition (SQL)

```
-- users
id (UUID), email, password_hash, name, is_guest, created_at, updated_at

-- auth_access_tokens
id, tokenable_id (FK: users), type, name, hash, session_id, created_at, updated_at,
last_used_at, expires_at

-- conversations
id (UUID), type (ENUM: direct, group), name, avatar_url, created_by (FK: users), created_at,
updated_at

-- conversation_participants
id, conversation_id (FK: conversations), user_id (FK: users), role (ENUM: admin, member),
last_read_at, created_at, updated_at

-- messages
id (UUID), conversation_id (FK: conversations), sender_id (FK: users), content, is_edited,
created_at, updated_at

-- notifications
id (UUID), user_id (FK: users), type, data (JSONB), read_at, created_at, updated_at

-- uploads
```

```
id (UUID), user_id (FK: users), file_name, original_name, mime_type, size, url, created_at,  
updated_at
```